Partial Channel Network: Compute Fewer, Perform Better

Haiduo Huang, Tian Xia, Wenzhe zhao, Pengju Ren
Xi'an Jiaotong University · Institute of Artificial Intelligence and Robotics
huanghd@stu.xjtu.edu.cn, stian_xia@xjtu.edu.cn, wenzhe@xjtu.edu.cn, pengjuren@xjtu.edu.cn

Abstract

Designing a module or mechanism that enables a network to maintain low parameters and FLOPs without sacrificing accuracy and throughput remains a challenge. To address this challenge and exploit the redundancy within feature map channels, we propose a new solution: **p***artial* **c***hannel* **m***echanism (***PCM***). Specifically, through the split operation, the feature map channels are divided into different parts, with each part corresponding to different operations, such as convolution, attention, pooling, and identity mapping. Based on this assumption, we introduce a novel* **p***artial* **at***tention* **conv***olution (***PATConv***) that can efficiently combine convolution with visual attention. Our exploration indicates that the PATConv can completely replace both the regular convolution and the regular visual attention while reducing model parameters and FLOPs. Moreover, PATConv can derive three new types of blocks: Partial Channel-Attention block (PAT_ch), Partial Spatial-Attention block (PAT_sp) and Partial Self-Attention block (PAT_sf). In addition, we propose a novel* **d***ynamic* **p***artial* **conv***olution (***DPConv***) that can adaptively learn the proportion of split channels in different layers to achieve better trade-offs. Building on PATConv and DPConv, we propose a new hybrid network family, named* **PartialNet***, which achieves superior top-1 accuracy and inference speed compared to some SOTA models on ImageNet-1K classification and excels in both detection and segmentation on the COCO dataset. Our code is available at* https://github.com/haiduo/PartialNet.

1. Introduction

Designing an efficient and effective neural network has remained a prominent topic in computer vision research. To design an efficient network, many prior works adopt depthwise separable convolution (DWConv) [13] as a substitute for regular dense convolution. For instance, some CNN-based models [27, 30] leverage DWConv to reduce the model's FLOPs and parameters, while Hybrid-based models [11, 26, 37] employ DWConv to simulate self-attention operations to decrease computation complexity. However, some studies [5, 20] have revealed that DWConv may suffer from frequent memory access and low parallelism during inference [3], which leads to low throughput.

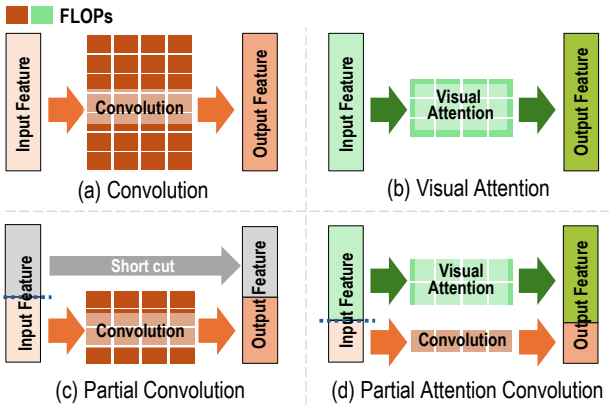


Figure 1. Comparison of different operation types.

Considering the substantial redundancy between feature maps [3, 8], to further reduce the computational cost and parameters while improving the model's inference speed and accuracy, we introduce the partial channel mechanism (PCM) to fully exploit the value of feature map channels. PCM primarily splits the feature maps into different parts by a split operation, with each part undergoing different operations, followed by an integration by a concatenation operation. Our insight is that by reasonably allocating different operations and using cheaper, efficient operators to partially replace costly, dense operators, it is possible to improve the inference speed of a model while further enhancing its accuracy. Specifically, our approach involves replacing computationally intensive convolutions with computationally cheaper visual attention to enhance the model's representation capability and improve inference speed. Based on this idea, we introduce a novel Partial Attention Convolution (PATConv), which can replace both regular convolutions and regular visual attention while also reducing the model's parameters and FLOPs. This approach primarily integrates partial convolution with partial visual attention,

as illustrated in Fig. 1.

Some recent works split the feature map, but they use a naive approach, where one part is operated on while the other part is left untouched. For instance, ShuffleNet V2 [20] introduces a "Channel Split" operation that divides the input feature channels into two parts: one part is retained directly, while the other part undergoes multiple layers of convolution. Similarly, FasterNet [3] introduces a partial convolution (PConv) that selectively applies Conv to a subset of input channels, leaving the remaining channels untouched. This makes PConv faster in inference speed compared to both regular Conv and regular DWConv. However, they merely consider improving the model's performance from the perspective of reducing the number of computed feature channels, neglecting the potential value of other channel portions.

Therefore, we believe that it is more reasonable to account for the overall FLOPs, inference speed, and accuracy from a global perspective by exploiting the entire feature channels. Specifically, we propose to combine Conv and attention, applying them to partial channels respectively. Since there are no effective attention algorithms for partial channels, we invent three efficient partial visual attention blocks: (1) PAT_ch integrates an enhanced Gaussian channel attention mechanism [14], facilitating richer inter-channel global information interaction. (2) PAT_sp introduces the spatial-wise attention to the MLP layer to further improve model accuracy. (3) PAT_sf refers to the MetaFormer-based [38] paradigm and integrate global self-attention into the last stage of the model to expand its global receptive field.

Although the above improvements can provide a stronger visual model, considering the limitations of latency and parameters, the ratio of split channels between partial convolution and partial visual attention needs to be further optimal. To address this problem, we refer to dynamic group convolution [39] and propose a novel dynamic partial convolution, which can effectively and adaptively learn the split ratio of different layers according to the constraints (*e.g.*, parameters and latency) to achieve the optimal trade-off between model inference speed and accuracy.

In conclusion, the enhanced model is dubbed **PartialNet**, which achieves overall performance improvement in the ImageNet1K classification task while maintaining similar throughput, as is presented in Fig. 2. Our main contributions can be described as:

- We propose a partial channel mechanism and introduce a partial attention convolution (PATConv) that integrates visual attention into partial convolution in a parallel way, which differs from the series way of previous works and can improve model performance while increasing inference speed.
- Based on the PATConv, we develop three partial visual attention blocks: PAT_ch exhibits high potential as a replacement for regular convolution and DWConv, PAT_sp can effectively reinforce MLP layers at minimal cost, while PAT_sf integrates local and global features, achieving higher accuracy.
- To achieve a better trade-off between model inference speed and accuracy, we propose a novel dynamic partial convolution (DPConv), which can adaptively learn the split ratio of different layers according to the constraints (*e.g.*, model parameters).
- Building upon the above methods, we design a new hybrid-based model family named PartialNet that shows improved performance on standard vision benchmarks over most efficient SOTA models.

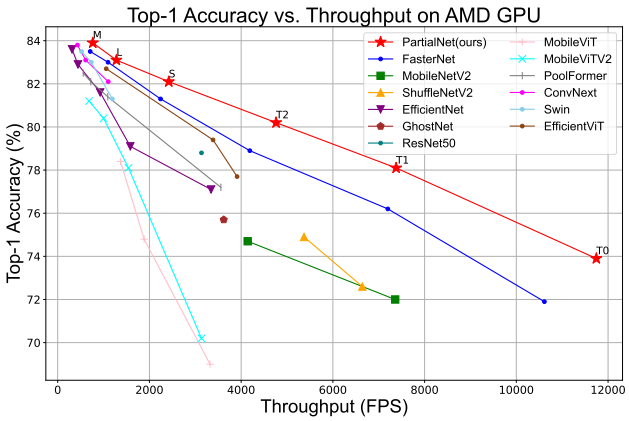


Figure 2. Our PartialNet achieves higher trade-off of accuracy and throughput on ImageNet-1K.

2. Related Work

Efficient CNNs and ViTs. DWConv is widely adopted in the design of efficient neural networks, such as MobileNets [12, 27], EfficientNets [30, 31], MobileViT [21], and EdgeViT [23]. Because of its efficiency limitations on modern parallel devices, numerous works have aimed to improve it. For example, RepLKNet [5] uses larger-kernel DWConv to alleviate the issue of underutilized calculations. PoolFormer [38] achieves strong performance through spatial interaction with pooling operations alone. Recently, FasterNet [3] reduces FLOPs and memory accesses simultaneously by introducing partial convolution. Nevertheless, FasterNet does not outperform other vision models in accuracy. In contrast, our proposed PartialNet addresses this limitation by integrating the visual attention into convolution to enhance the accuracy of models.

Visual Attention. The effectiveness of Vision Transformers (ViTs) mainly attributes their success to the role of attention mechanisms [24, 25]. In visual tasks, attention mechanisms are commonly categorized into three types:

Channel Attention, Spatial Attention, and Self-Attention. Some works [2, 22, 26, 29] employ various techniques to implement the Self-Attention mechanism efficiently, *e.g.*, Linear Attention [2, 32]. Furthermore, the effectiveness of Channel Attention and Spatial Attention has already been validated in SRM [17], SE-Net [14] and CBAM [34]. Similarly, we have incorporated attention to the same feature, but in a parallel way with partial attention to mitigate the impact of element-wise multiplication on overall inference speed.

3. Methodology

In this section, we start by introducing the motivation behind enhancing partial convolution with visual attention and present our novel Partial Attention Convolution (PATConv) mechanism, which leverages attention on a subset of feature channels to balance computational efficiency and accuracy. Next, we detail three innovative blocks within PATConv: the Partial Channel-Attention block (PAT_ch), which integrates Conv3×3 with channel attention for global spatial interaction; the Partial Spatial-Attention block (PAT_sp), which combines Conv1×1 with spatial attention to efficiently mix channel-wise information; and the Partial Self-Attention block (PAT_sf), which applies self-attention selectively to extend the model's receptive field. We further introduce a learnable dynamic partial convolution (DPConv) with adaptive channel split ratio for improved model flexibility. Finally, we describe the overall PartialNet architecture, structured in four hierarchical stages with PATConv-integrated blocks, aimed at achieving a robust speed-accuracy trade-off across model variants.

3.1. Partial Channel Mechanism

Generally, designing an efficient neural network necessitates comprehensive consideration and optimization from various perspectives, including fewer FLOPs, smaller model sizes, lower memory access, and better accuracy. Recently, some works (e.g., MobileViTv2 [22] and EfficientVit [2]) attempt to combine depthwise separable convolutions with the self-attention mechanism to reduce the model parameters and latency. Other works (e.g., ShuffleNetv2 [20] and FasterNet [3]) try to reduce FLOPs and improve inference speed by performing feature extraction using only a subset of the feature map channels. However, it does not exhibit a noticeable accuracy advantage when compared to models with similar parameters or FLOPs. Among, FasterNet only uses partial convolution, achieving exceptional speed across various devices. However, we find that FasterNet simply performs convolution operations on partial channels, which can reduce FLOPs and latency but leads to limited feature interaction and lack of global information exchange.

In contrast, we comprehensively exploit the potential value within the channels of the entire feature map. For different partials, we use different operations to further reduce the model FLOPs while improving accuracy. It can be called the partial channel mechanism. Based on this, we propose a new type of convolution that replaces computationally expensive dense convolution operations with cost-effective visual attention, called Partial Attention Convolution (PATConv). Previous research [3, 8] has demonstrated that redundancy exists among feature channels, making attention operations applied to partial channels a form of global information interaction.

Unlike regular visual attention methods, our PATConv is more efficient due to using only a subset of channels for the computationally expensive element-wise multiplication. Indeed, running two operations in parallel on separate branches allows for simultaneous computation, optimizing resource utilization on the GPUs [16]. Suppose the input and output of our PATConv is denoted as $F \in \mathbb{R}^{h\times w\times c_{in}}$ and $O \in \mathbb{R}^{h\times w\times c_{out}}$ respectively, where c_{in} and c_{out} represent the number of input and output channels, h, w is the height and width of a channel, respectively. Suppose $c_{in} = c_{out}$, PATConv can be defined as

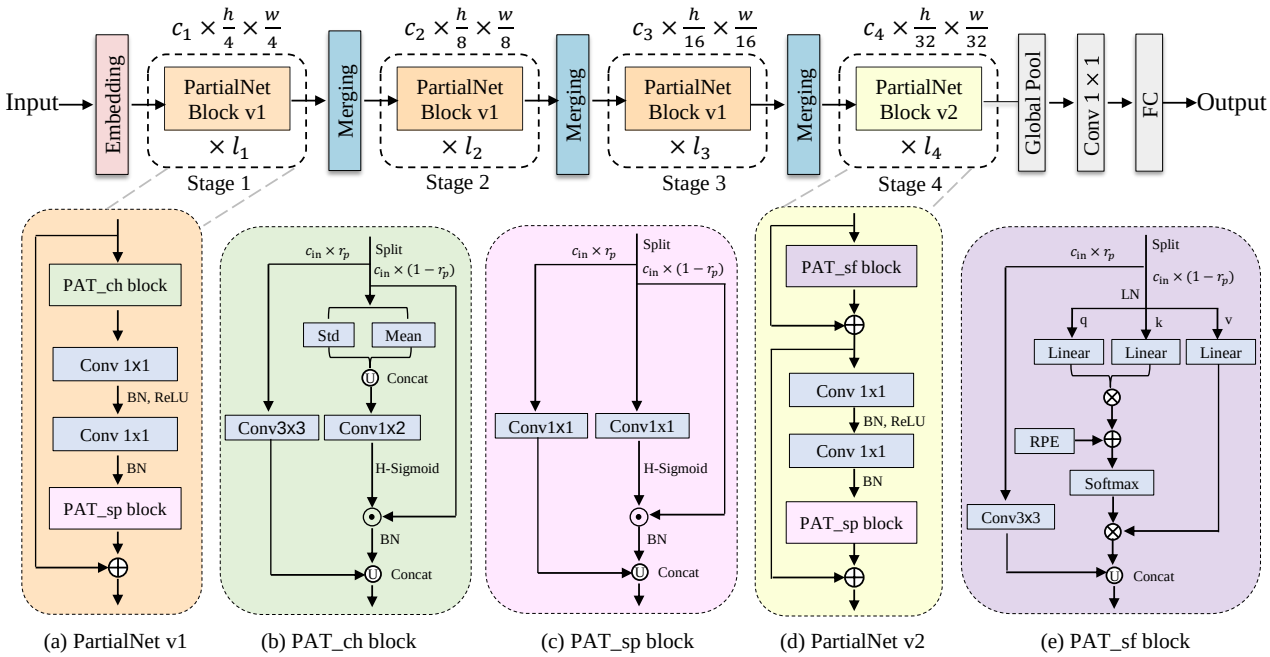
$$O = \text{PATConv}(F) = \text{Conv}(F^{c_{in}\times r_p}) \cup \text{Atten}(F^{c_{in}(1-r_p)}) \quad (1)$$

where the symbol $\cup$ and Atten denote the concatenation operation and the visual attention operation respectively. The r_p is a hyperparameter representing the split ratio of the channels and can be learned adaptively.

In addition, PATConv can apply channel-wise and spatial-wise mixing to enhance global information and integrate self-attention mechanisms to expand the model's receptive field to derive three blocks, proving to be highly effective.

PAT_ch: We first propose to integrate Conv3×3 and channel attention involving global spatial information interaction, and using an enhanced Gaussian-SE module compute channels' mean and variance to squeeze global spatial information. Unlike SENet [14], it only considers the mean information of the channel and ignores the statistical information of std. Considering that the feature maps obey an approximately normal distribution [15] during training, we fully utilize the Gaussian statistical to express the channel-wise representation information, as shown in Fig. 3 (b).

PAT_sp: Secondly, we integrate spatial attention with Conv1×1 because both operations mix channel wise information. Our spatial attention employs a point-wise convolution to squeeze global channel information into tensor with only one channel. After passing through a Hard-Sigmoid activation, this tensor serves as the spatial attention map to weight features. We position PAT_sp after the MLP layer, enabling the Conv1×1 component of PAT_sp to merge with the second Conv1×1 in the MLP layer during inference, as shown in Fig. 3 (c). This setup further minimizes the impact of attention on inference speed, and its details of the merge



(a) PartialNet v1 (b) PAT_ch block (c) PAT_sp block (d) PartialNet v2 (e) PAT_sf block

Figure 3. The overall architecture of our PartialNet, consisting of four hierarchical stages, each incorporating a series of PartialNet blocks followed by an embedding or merging layer. The last three layers are dedicated to feature classification. Where $\odot$ and $\otimes$ denote element-wise multiplication and matrix multiplication respectively.

operation refer to Fig. 1 (d) in the appendix.

PAT_sf: Finally, self-attention not only engages with spatial information interaction but also extends the model's effective receptive field, which can replace channel attention. However, because the computational complexity of self-attention operations increases quadratically with the size of the feature map, we restrict the use of PAT_sf to the last stage to achieve a superior speed-accuracy trade-off. Beside, we employee relative position encoding (RPE) [35] into the attention map, which can further enhances model accuracy, as shown in Fig. 3 (e).

Notably, unlike conventional CNNs combined with attention, which process steps one after the other, we process steps simultaneously on the same input, improving the balance between speed and accuracy. Moreover, our PATConv is not limited to the above three combinations, it can be efficiently combined with more visual attention modules.

3.2. Learnable Dynamic Partial Convolution

For the PATConv, the split ratio r_p is a critical hyperparameter that significantly influences the parameters and latency of a model. A too-large r_p causes PATConv to degenerate into a regular convolution, rendering the visual attention component ineffective at capturing global information. Conversely, a too-small r_p results in PATConv lacking essential local inductive bias information.

Achieving higher accuracy and throughput at similar complexity often necessitates extensive experimentation to identify an optimal r_p. In FasterNet, a default split ratio of 1/4 is for all variants. In contrast, we propose a dynamic partial convolution (DPConv) in which the r_p is learnable. This approach allows a model to adaptively determine the optimal r_p for different layers during training. The strategies can be modeled by a binary relationship matrix $U \in \{0,1\}^{c_{in} \times c_{out}}$ [39]. The entire matrix U can be decomposed in into a set of K small matrix $U_k \in \{0,1\}^{2\times 2}$, where U_k either equal to a 2-by-2 constant matrix of ones $\mathbf{1}$ or equal to 2-by-2 identity matrix I, *i.e.*,

$$U = U_1 \circledast U_2 \circledast ... \circledast U_K \tag{2}$$

$$U_k = g_k \mathbf{1} + (1 - g_k) I, \ \forall g_k \in g, \ g = \text{Sign}(\widetilde{g}) \tag{3}$$

where $\circledast$ denotes a Kronecker product [1] and $\widetilde{g} \in \mathbb{R}^K$ is a learnable gate vector taking continues value, and $g \in \{0,1\}^K$ is a binary gate vector derived from $\widetilde{g}$. Since the Sign function is not differentiable, the gate parameters are optimized using a straight-through estimator [4], similar to the binary network quantization method, to ensure convergence. Suppose $c_{in} = c_{out}$, so $K = \log_2 c_{in}$. DPConv can be defined as

$$O = \text{DPConv}(F) = U \odot \mathbf{m} \odot W \tag{4}$$

where $\odot$ denotes elementwise product, the $\mathbf{m} \in \mathbb{R}^{c_{in}}$ is a vector to mask the useless part of U. Since the result of the Kronecker product is a power of 2 matrix, the number of channels in each layer of the model also needs to be 2^K.

So, it is not hard to deduce that

$$r_p = \frac{2^{\sum_{k=1}^{K}(1-g_k)}}{c_{in}}, \ \mathbf{m}_i = \begin{cases} 1 & \text{if } i < \frac{1}{r_p} \\ 0 & \text{if } i \geq \frac{1}{r_p} \end{cases} \tag{5}$$

where the g_k indicates the k-th component of g. The specific generation process of DPConv is shown in Fig. 4.

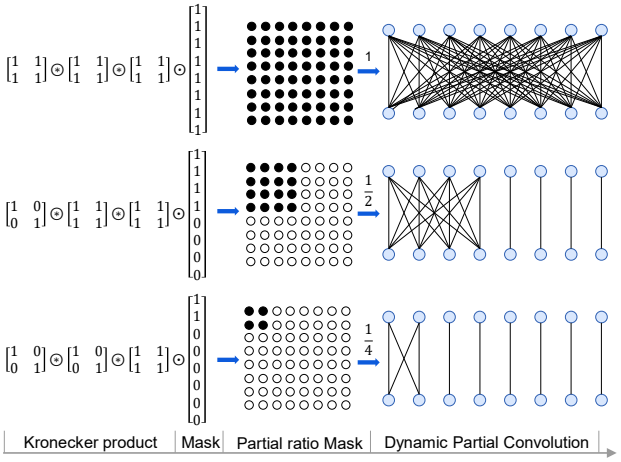


Figure 4. The generation process of DPConv, where $\odot$ denotes elementwise product, $\circledast$ denotes a Kronecker product.

Considering the constraints of model deployment, *e.g.*, the parameters and FLOPs, it is necessary to limit the learned r_p to avoid being too large. Therefore, we design a resource-constrained training scheme for DPConv. To simplify the calculation, assuming that we only consider the case of partial convolution. We propose a regularization term denoted as ζ to constrain the computational complexity by

$$\zeta = \sum_{l=1}^{L} \zeta_l, \ \zeta_l = \sum_{i=1}^{c_{in}} \sum_{j=1}^{c_{out}} u_{ij}, \forall u_{ij} \in U \tag{6}$$

where L denotes the number of DPConv layers, and u_{ij} denotes an element of U. The term ζ_l represents the number of non-zero elements in U, measuring the number of activated convolution weights of the l-th DPConv layer. Thus, ζ can be treated as a measurement of the model's computational complexity. In fact, it can be deduced that the sum of each row or each column of U can be calculated as $\prod_{k=1}^{K}(1+g_k)$. Substituting it to Eq. (6) gives us

$$\zeta = \sum_{l=1}^{L}(\prod_{k=1}^{K^l}(1+g_k^l) \cdot \prod_{k=1}^{K^l}(1+g_k^l)) = \sum_{l=1}^{L} 2^{2 \cdot \sum_{k=1}^{K^l} g_k^l} \tag{7}$$

where g_k^l and K^l indices g_k and K in the l-th layer, respectively. Here we suppose $c_l = c_{in} = c_{out}$. Let $\kappa = \sum_{l=1}^{L} \frac{c_l}{\theta}^2$ represent the desire computational complexity of the entire network. By setting θ, we can control the overall complexity of the PartialNet. For example, when θ=4, which is equal to the complexity of r_p=1/4 in FasterNet. So, a weighted product $[\frac{\kappa}{\zeta}]^\alpha$ to approximate the Pareto optimal problem, α is a constant value by empirically set. And we have $\alpha = 0$ if $\zeta \leq \kappa$, implying that the complexity constraint is satisfied. Otherwise, $\alpha = -0.01$ is used to penalize the model complexity when $\zeta > \kappa$.

Additionally, the split ratio mask is closely related to the gate vector g. A reasonable Kronecker product can only be generated when the elements in g are ordered such that all elements of I come before those of $\mathbf{1}$ for DPConv. Therefore, it is necessary to constrain g by incorporating a regularization loss term ψ, which can be computed by

$$\psi = \sum_{l=1}^{L} \psi_l, \ \psi_l = \begin{cases} \sum_i^K |\widetilde{g}_k^l| & \text{if } g_i^l = 1 \text{ and } g_{i+1}^l = 0, \\ 0 & \text{otherwise.} \end{cases} \tag{8}$$

Finally, our objective is to search a PartialNet model that

$$\begin{cases} \text{minimize} & \mathcal{L}(\{w_l\}_{l=1}^{L}, \{\widetilde{g}_l\}_{l=1}^{L}) \cdot [\frac{\kappa}{\zeta}]^\alpha + \psi\beta, \\ \text{subject to} & \zeta \leq \kappa. \end{cases} \tag{9}$$

where $\beta \in (0, 1]$ is the penalty factor of ψ, default is 0.9.

3.3. PartialNet Architecture

The overall architecture of PartialNet is depicted in Fig. 3, consists of four hierarchical stages, each of which precedes an embedding layer (a regular Conv4×4 with stride 4) or a merging layer (a regular Conv2×2 with stride 2). These layers serve for spatial downsampling and channel number expansion. Each stage comprises a set of PartialNet blocks. In the first three stages of the PartialNet, we employ "PartialNet Block v1" including PAT_ch block and PAT_sp block, as shown in Fig. 3 (a). Similarly, we employ "PartialNet Block v2" by replacing PAT_ch with PAT_sf in the last stage and modifying the shortcut connection way to achieve stable training, as shown in Fig. 3 (d).

In addition, we maintain normalization or activation layers only after each intermediate Conv1×1 to preserve feature diversity and achieve higher throughput. We also incorporate batch normalization into adjacent Conv layers to expedite inference without sacrificing performance. For the activation layer, the smaller PartialNet variants uses GELU, while the larger PartialNet variants employs ReLU. The last three layers consist of global average pooling, Conv1×1, and a fully connected layer. These layers collectively serve for feature transformation and classification. We offer tiny, small, medium, and large variants of PartialNet, which are denoted as PartialNet-T0/1/2, PartialNet-S, PartialNet-M, and PartialNet-L. These variants share a similar architecture but differ in depth and width. For detailed specifications please refer to Tab. 3 of the appendix.

4. Experiments

4.1. PartialNet on ImageNet-1k Classification

Setup. ImageNet-1K is one of the most extensively used datasets in computer vision. It encompasses 1K common

Network	Type	Params (M)↓	FLOPs (G)↓	TP V100 (FPS)↑	TP MI250 (FPS)↑	Latency CPU (ms)↓	Top-1 (%)↑
ShuffleNetV2 x1.5[20]	CNN	3.5	0.30	5315	6642	13.7	72.6
MobileNetV2[27]	CNN	3.5	0.31	3924	7359	13.7	72.0
FasterNet-T0[3]	CNN	3.9	0.34	**8546**	10612	**10.5**	71.9
MobileViTv2-0.5[22]	Hybrid	1.4	0.46	3094	3135	15.8	70.2
PartialNet-T0(ours)	Hybrid	4.3	0.25	7777	**11744**	12.2	**73.9**
EfficientNet-B0[30]	CNN	5.3	0.39	2934	3344	22.7	77.1
ShuffleNetV2 x2[20]	CNN	7.4	0.59	4290	5371	22.6	74.9
MobileNetV2 x1.4[27]	CNN	6.1	0.60	2615	4142	21.7	74.7
FasterNet-T1[3]	CNN	7.6	0.85	**4648**	7198	22.2	76.2
PartialNet-T1(ours)	Hybrid	7.8	0.55	4403	**7379**	**21.5**	**78.1**
EfficientNet-B1[30]	CNN	7.8	0.70	1730	1583	35.5	79.1
ResNet50[9]	CNN	25.6	4.11	1258	3135	94.8	78.8
FasterNet-T2[3]	CNN	15.0	1.91	2455	4189	43.7	78.9
PoolFormer-S12[38]	Hybrid	11.9	1.82	1927	3558	56.1	77.2
MobileViTv2-1.0[22]	Hybrid	4.9	1.85	1391	1543	41.5	78.1
EfficientViT-B1[2]	Hybrid	9.1	0.52	3072	3387	**25.7**	79.4
PartialNet-T2(ours)	Hybrid	12.6	1.03	**3074**	**4761**	35.2	**80.2**
EfficientNet-B3[30]	CNN	12.0	1.80	768	926	73.5	81.6
FasterNet-S[3]	CNN	31.1	4.56	1261	2243	96.0	81.3
PoolFormer-S36[38]	Hybrid	30.9	5.00	675	1092	152.4	81.4
MobileViTv2-2.0[22]	Hybrid	18.5	7.50	551	684	103.7	81.2
Swin-T[18]	Hybrid	28.3	4.51	808	1192	107.1	81.3
PartialNet-S(ours)	Hybrid	29.0	2.71	**1559**	**2422**	**72.5**	**82.1**
EfficientNet-B4[30]	CNN	19.0	4.20	356	442	156.9	82.9
FasterNet-M[3]	CNN	53.5	8.74	621	1098	181.6	83.0
PoolFormer-M36[38]	Hybrid	56.2	8.80	444	721	244.3	82.1
Swin-S[18]	Hybrid	49.6	8.77	477	732	199.1	83.0
PartialNet-M(ours)	Hybrid	61.3	6.69	**799**	**1280**	**155.3**	**83.1**
EfficientNet-B5[30]	CNN	30.0	9.90	246	313	333.3	83.6
ConvNeXt-B[19]	CNN	88.6	15.38	322	430	317.1	83.8
FasterNet-L[3]	CNN	93.5	15.52	384	709	312.5	83.5
PoolFormer-M48[38]	Hybrid	73.5	11.59	335	556	322.3	82.5
Swin-B[18]	Hybrid	87.8	15.47	315	520	333.8	83.5
PartialNet-L(ours)	Hybrid	104.3	11.91	**426**	**765**	**272.5**	**83.9**

Table 1. Comparison on ImageNet-1K Benchmark: Models with similar top-1 accuracy are grouped together. The TP denotes throughput.

classes, consisting of approximately 1.3M training images and 50K validation images. We train our models on the ImageNet-1k dataset for 300 epochs using AdamW optimizer with 20 epochs linear warm-up. And we use the same regularization and augmentation techniques and multi-scale training as FasterNet. For detailed experimental settings please refer to Tab. 2 of the appendix. For inference speed, we test the model's throughput in Nvidia-V100 and AMD-Instinct-MI250 GPUs with batch size of 256, we test latency in AMD EPYC^{TM} 73F3 CPU with one core.

Results. Tab. 1 provides a comparison of our PartialNet models (T0, T1, T2, S, M, and L) with previous SOTA cnn-based and hybrid-based models. The experimental results demonstrate that PartialNet consistently surpasses recent models like FasterNet [3] across all model variants. For example, PartialNet-T2 achieves 1.3% higher accuracy than FasterNet-T2 while exhibiting around 25.2%(or 13.7%) increase in V100(or MI250) throughput and 24.1% lower CPU latency. The results demonstrate that the combination of partial visual attention and partial convolution significantly improves model performance while increasing throughput. There is a certain difference in hardware architecture between the V100 and MI250 because the V100 is better suited for compute-intensive tasks, while the MI250 excels in bandwidth-intensive tasks [7]. This may explain why our PartialNet has slightly lower throughput than FasterNet in the T0 and T1 variants on the V100. For more comparison please refer to Tab. 4 of the appendix.

In addition, it can be seen from Fig. 5, as our PartialNet variants gradually increase from T0 to L, both Top-1 accuracy and FLOPs increase, but the improvement in Top-1 accuracy relative to the increase in FLOPs is more pronounced. This further demonstrates that our PartialNet achieves better accuracy with less computational cost.

4.2. PartialNet on Downstream Tasks

Setup. We utilize the pre-trained PartialNet as the backbone within the Mask-RCNN [10] detector for object detection

Backbone	Params (M)↓	FLOPs (G)↓	Throughput (FPS)↑	AP^b ↑	AP^b_{50} ↑	AP^b_{75} ↑	AP^m ↑	AP^m_{50} ↑	AP^m_{75} ↑
ResNet50[9]	44.2	253	121	38.0	58.6	41.4	34.4	55.1	36.7
PoolFormer-S24[38]	41.0	233	68	40.1	62.2	43.4	37.0	59.1	39.6
PVT-Small x1.5[33]	44.1	238	98	40.4	62.9	43.8	37.8	60.1	40.3
FasterNet-S[3]	49.0	258	121	39.9	61.2	43.6	36.9	58.1	39.7
PartialNet-S(ours)	46.9	216	**122**	**42.7**	**64.9**	**46.5**	**39.3**	**61.8**	**42.2**
ResNet101[21]	63.2	329	62	40.4	61.1	44.2	36.4	57.7	38.8
ResNeXt101-32×4d[36]	62.8	333	51	41.9	62.5	45.9	37.5	59.4	40.2
PoolFormer-S36[38]	50.5	266	44	41.0	63.1	44.8	37.7	60.1	40.0
PVT-Medium[33]	63.9	295	52	42.0	64.4	45.6	39.0	61.6	42.1
FasterNet-M[3]	71.2	344	62	43.0	64.4	47.4	39.1	61.5	42.3
PartialNet-M(ours)	78.2	295	**65**	**44.3**	**65.8**	**48.5**	**40.6**	**63.3**	**43.7**
ResNeXt101-64×4d[36]	101.9	487	29	42.8	63.8	47.3	38.4	60.6	41.3
PVT-Large×4d[33]	81.0	358	26	42.9	65.0	46.6	39.5	61.9	42.5
FasterNet-L[3]	110.9	484	35	44.0	65.6	48.2	39.9	62.3	43.0
PartialNet-L(ours)	122.0	397	**39**	**44.7**	**66.3**	**49.0**	**41.0**	**63.7**	**44.2**

Table 2. Results using PartialNet-S/M/L on object detection and instance segmentation benchmark in COCO dataset.

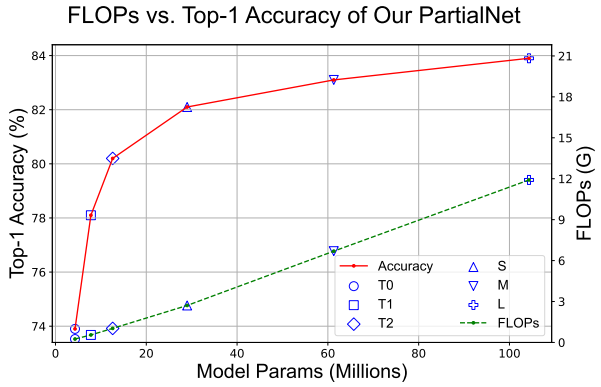


Figure 5. The relationship between FLOPs and Top-1 Accuracy in different PartialNet model variants.

and instance segmentation on the MS-COCO 2017 dataset, comprising 118K training images and 5K validation images. To highlight the effectiveness of the backbone itself, we follow the FasterNet approach and employ the AdamW optimizer, conduct training of 12 epochs, use a batch size of 16, image size of 1333×800, and maintain other training settings without further hyperparameter tuning.

Results. Tab. 2 presents a comparison of PartialNet with representative models, reporting performance in terms of average precision (mAP) for both detection and instance segmentation. The results show that PartialNet consistently outperforms mainstream SOTA models, achieving higher Average Precision (AP) while maintaining similar inference speed. For instance, PartialNet-S achieves over 7% higher AP^b in detection metrics and 6.5% higher AP^m in segmentation metrics with fewer parameters and FLOPs compared to FasterNet-S, while maintaining similar throughput. The results further confirm the generalization capabilities of our proposed PartialNet across various tasks.

4.3. Ablation Studies

Partial Attention vs. Full Attention. To prove the superiority of our PATConv over full attention, we conduct comparative experiments on the PartialNet-T2, as shown in Tab. 3. Specifically, we replace PATConv with corresponding regular full attention for comparison. Full attention involves conducting visual attention calculations on all channels of the input feature map, which is the common way of conventional visual attention mechanism. The results indicate that our PATConv achieves a superior balance between inference speed and performance compared to the full attention counterpart. In addition, we adopt Grad-CAM [28] to visualize the attention. Results in Fig. 6 show that partial visual attention can focus on the target objects. It is feasible to perform attention operations on part channels and confirm the effectiveness of our improved partial channel attention mechanism.

ch-sp-sf	Params (M)	FLOPs (G)	Throughput (FPS)↑	Latency (ms)↓	Top-1 (%)↑
P-P-P	12.6	1.03	**4761**	**35.2**	**80.2**
F-P-P	13.0	1.04	4662	36.5	80.1
P-F-P	12.6	1.04	4688	35.6	79.9
P-P-F	14.5	1.12	4600	38.6	**80.2**

Table 3. Comparison on PartialNet-T2 of partial attention (P), and full attention (F) on ImageNet-1K dataset. Where the "ch", "sp", and "sf" denote channel-wise attention, spatial-wise attention, and self-attention respectively.

Effect of three PATConv blocks. To confirm the individual effects of our proposed three PATConv blocks, we conducted ablation studies by progressively adding each block one by one, as indicated in Tab. 4. The results indicate that the three proposed PATConv blocks consistently enhance model performance. Additionally, Tab. 5 also pro-

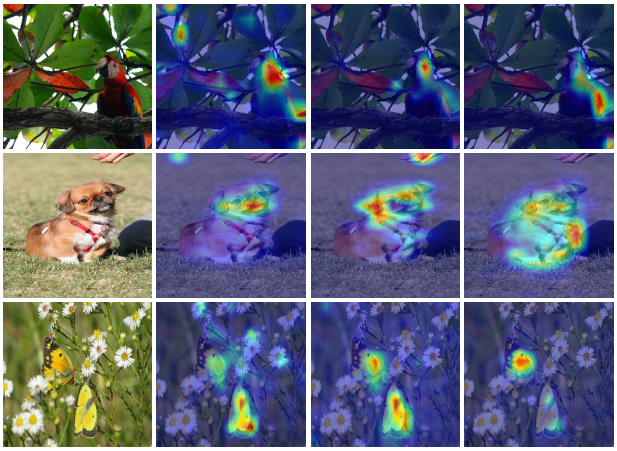


Figure 6. Visualization results show different categories of the ImageNet-1K validation set using Grad-CAM.

vides a reproduced comparison of the PATConv applied to other models. The results further demonstrate the effectiveness of our proposed PATConv blocks. The training process of ConvNext-tiny is shown in Fig. 2 of the appendix.

ch	sp	sf	Params (M)	FLOPs (G)	Throughput (FPS)↑	Latency (ms)↓	Top-1 (%)↑
w/o	w/o	w/o	11.1	0.92	**6405**	**25.7**	76.0
w.	w/o	w/o	11.1	0.92	5440	30.9	77.4
w.	w.	w/o	11.5	0.92	5157	31.7	78.9
w.	w.	w.	12.6	1.03	4761	35.2	**80.2**

Table 4. Ablation experiments of PartialNet-T2 with different configurations of PAT blocks on the ImageNet-1K.

Model	Throughput(FPS)↑		Top-1(%)↑	
	original	PATConv	original	PATConv
ResNet50	1258	2832	76.13	77.64
MobileNetV2	3924	4560	71.14	73.85
ConvNext-tiny	902	1123	76.00	78.57

Table 5. The results of applying PATConv to other models.

PATConv vs. Regular Conv and DWConv. To further verify the advantages of our proposed partial attention convolution, such as PAT_ch, over regular convolution (Conv) and DepthWise convolution (DWConv), we conduct ablation experiments on PartialNet-T2, as is shown in Tab. 6. To make a fair comparison, we widen DWConv to keep the throughput of the three convolution types in the same range. The results show that our proposed PAT_ch surpasses Conv and DWConv in all metrics including Params, Flops, throughput, latency and Top-1 accuracy, which validates the efficiency and effectiveness of PATConv.

Conv3×3	Params (M)	FLOPs (G)	Throughput (FPS)↑	Latency (ms)↓	Top-1 (%)↑
PAT_ch	**12.6**	**1.03**	**4761**	**35.2**	**80.2**
Conv	15.8	2.12	4190	49.9	79.9
DWConv	15.8	1.28	4017	35.4	79.6

Table 6. Ablation on PartialNet-T2 with different convolution types on ImageNet-1K dataset.

Different Constraints Settings. For different constraints θ and other parameters fixed, we conduct different experiments on PartialNet-T0, as is shown in Fig. 7. We find that the learned split ratio r_p across different layers exhibit a consistent pattern: as the θ increase, the first and last layers are less likely to exhibit sparsity compared to the middle layers, which is consistent with the conclusion of model quantization [6], that the first and last layers of the model are more important. For detailed specifications of split ratio r_p across different layers please refer to Tab. 3 of the appendix.

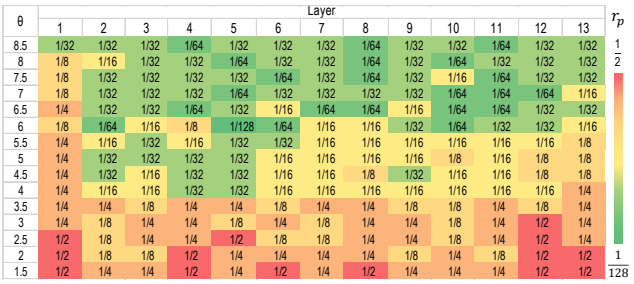


Figure 7. The learned split ratios r_p of different layers of PartialNet-T0 under different complexity constraints θ.

5. Conclusion

Feature selection theory shows that there may be a certain degree of redundancy and correlation between features. While this redundancy does not provide additional information gain, it can increase computational complexity and heighten the risk of overfitting. Our research builds on this theory from an implementation perspective, achieving a balance of optimal performance and computational efficiency. Specifically, we introduce the partial channel mechanism and propose Partial Attention Convolution, which strategically integrates visual attention into the convolution process to enhance feature utility. Furthermore, we present Dynamic Partial Convolution, an adaptive approach that learns optimal split ratios for channels across different layers in the model. With these innovations, we develop the PartialNet architecture, which surpasses recent efficient networks on ImageNet-1K classification as well as COCO detection and segmentation tasks. This underscores the effectiveness of the partial channel mechanism in achieving an optimal balance between

accuracy and efficiency across a range of vision tasks.

References

[1] Bobbi Jo Broxson. The kronecker product. 2006. 4

[2] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Lightweight multi-scale attention for high-resolution dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17302–17313, 2023. 3, 6, 4

[3] Jierun Chen, Shiu-hong Kao, Hao He, Weipeng Zhuo, Song Wen, Chul-Ho Lee, and S-H Gary Chan. Run, don't walk: Chasing higher flops for faster neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12021–12031, 2023. 1, 2, 3, 6, 7, 4, 5

[4] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *arXiv preprint arXiv:1602.02830*, 2016. 4

[5] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Scaling up your kernels to 31x31: Revisiting large kernel design in cnns. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11963–11975, 2022. 1, 2

[6] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 293–302, 2019. 8

[7] Kamil Halbiniak, Norbert Meyer, and Krzysztof Rojek. Single- and multi-gpu computing on nvidia- and amd-based server platforms for solidification modeling application. *Concurrency and Computation: Practice and Experience*, 36 (9):e8000, 2024. 6

[8] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. Ghostnet: More features from cheap operations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1580–1589, 2020. 1, 3, 4

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. 6, 7, 4, 5

[10] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 6

[11] Qibin Hou, Cheng-Ze Lu, Ming-Ming Cheng, and Jiashi Feng. Conv2former: A simple transformer-style convnet for visual recognition. *arXiv preprint arXiv:2211.11943*, 2022. 1

[12] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for mobilenetv3. *Proceedings of the IEEE/CVF international conference on computer vision*, abs/1905.02244, 2019. 2

[13] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 1

[14] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018. 2, 3, 5

[15] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015. 3

[16] David B Kirk and W Hwu Wen-Mei. *Programming massively parallel processors: a hands-on approach*. Morgan kaufmann, 2016. 3

[17] HyunJae Lee, Hyo-Eun Kim, and Hyeonseob Nam. Srm: A style-based recalibration module for convolutional neural networks. In *Proceedings of the IEEE/CVF International conference on computer vision*, pages 1854–1862, 2019. 3, 5

[18] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 6, 4

[19] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022. 6, 4

[20] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*, pages 116–131, 2018. 1, 2, 3, 6, 4

[21] Sachin Mehta and Mohammad Rastegari. Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*, 2021. 2, 7, 4, 5

[22] Sachin Mehta and Mohammad Rastegari. Separable self-attention for mobile vision transformers. *arXiv preprint arXiv:2206.02680*, 2022. 3, 6, 4

[23] Junting Pan, Adrian Bulat, Fuwen Tan, Xiatian Zhu, Lukasz Dudziak, Hongsheng Li, Georgios Tzimiropoulos, and Brais Martinez. Edgevits: Competing light-weight cnns on mobile devices with vision transformers. In *European Conference on Computer Vision*, pages 294–311. Springer, 2022. 2

[24] Sayak Paul and Pin-Yu Chen. Vision transformers are robust learners. In *Proceedings of the AAAI conference on Artificial Intelligence*, pages 2071–2081, 2022. 2

[25] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in Neural Information Processing Systems*, 34:12116–12128, 2021. 2

[26] Yongming Rao, Wenliang Zhao, Yansong Tang, Jie Zhou, Ser Nam Lim, and Jiwen Lu. Hornet: Efficient high-order spatial interactions with recursive gated convolutions.